

Pyomo Implementation Pipeline for Huawei TechArena 2025

Gen's BESS Optimization Team

September 30, 2025

This document outlines the step-by-step process for implementing the BESS optimization model using Python and Pyomo.

1 Project Setup

1.1 Directory Structure

A logical project structure is crucial for maintainability.

```
1 .
2 |-- input/
3 |   |-- TechArena2025_data.jsonl
4 |-- output/
5 |-- main.py
6 |-- requirements.txt
```

Listing 1: Recommended Directory Structure

1.2 Environment and Dependencies

A dedicated virtual environment prevents dependency conflicts. The `requirements.txt` file should specify all necessary packages.

```
1 pandas
2 numpy
3 pyomo
4 openpyxl
5 # A solver like CBC or GLPK must be installed and accessible in the system PATH.
```

Listing 2: requirements.txt

2 Data Loading and Preprocessing (main.py)

This phase transforms raw data into a model-ready format.

2.1 Load Raw Data

- Read the `TechArena2025_data_tidy.jsonl` file line by line into a pandas DataFrame.

2.2 Clean and Structure Data

- Convert the `timestamp` column to datetime objects and set it as the DataFrame index.
- Pivot the DataFrame to create a multi-level column structure: (`country`, `source`, `direction`).

2.3 Resample and Align Time Series

- Resample all time series to a consistent 15-minute frequency (`15T`).
- Use forward-fill (`ffill`) to propagate prices within their validity periods (e.g., a 4-hour ancillary price applies to all 15-min intervals within that block).
- Create helper columns for 'day' and '4-hour block' identifiers to facilitate constraint mapping.

3 Main Loop Structure (main.py)

The core logic iterates through every required scenario (country $\times$ C-rate $\times$ cycle limit).

```
1 countries = ['DE', 'AT', 'CH', 'HU', 'CZ']
2 c_rates = [0.25, 0.33, 0.50]
3 daily_cycles = [1.0, 1.5, 2.0]
4
5 all_scenario_results = {}
6
7 for country in countries:
8     for c_rate in c_rates:
9         for n_cycles in daily_cycles:
10             # Get the preprocessed data for the current country
11             country_data = ...
12
13             # Define BESS parameters for this specific scenario
14             bess_params = {
15                 'E_nom': 4472.0, # kWh [1]
16                 'P_max_config': 4472.0 * c_rate, # kW [1]
17                 'N_cycles': n_cycles,
18                 'eta_ch': 0.90*0.5, # Round-trip efficiency is 90%
19                 'eta_dis': 0.90*0.5,
20                 'SOC_min': 0.0,
21                 'SOC_max': 1.0,
22                 'delta_t': 0.25, # hours
23                 'delta_b': 4.0, # hours
24                 'min_bid_fcr': 1.0, # MW
25                 'min_bid_afrr': 1.0 # MW
26             }
27
28             # Apply country-specific rules (CRITICAL)
29             if country == 'CH':
30                 # Swiss aFRR market has a 5 MW minimum bid, BESS max power is ~2.2 MW
31                 # Therefore, the BESS cannot participate in the Swiss aFRR market.
32                 bess_params['can_bid_afrr'] = False
33             else:
34                 bess_params['can_bid_afrr'] = True
35
36             # Call the optimization function
37             # This should implement the rolling horizon strategy (see Section 6)
38             solution = solve_bess_optimization(country_data, bess_params)
39
40             # Store the results
41             all_scenario_results[(country, c_rate, n_cycles)] = solution
42
43 # 5. Post-process all_results and generate output files
```

Listing 3: Pseudocode for the Main Scenario Loop

4 Pyomo Model Implementation (run_optimization function)

This function encapsulates the mathematical model definition and solver execution. Follow the official Pyomo documentation for detailed syntax and capabilities: <https://pyomo.readthedocs.io/en/stable/>.

4.1 Initialize Model and Define Sets

```
1 import pyomo.environ as pyo
2
3 def run_optimization(data, params):
4     model = pyo.ConcreteModel(name="BESS_Optimization")
5
6     # Define Sets from data indices
7     model.T = pyo.Set(initialize=range(len(data.index)))
8     model.B = pyo.Set(initialize=data['block_id'].unique())
9     model.D = pyo.Set(initialize=data['day_id'].unique())
```

4.2 Define Parameters and Variables

- Create `pyo.Param` objects for all input data (prices, efficiencies, etc.).
- Create `pyo.Var` objects for all decision variables, specifying domains (e.g., `pyo.NonNegativeReals`, `pyo.Binary`).

4.3 Define Objective Function and Constraints

Each mathematical equation is translated into a Python function.

```
1 # --- Objective Function ---
2 def objective_rule(model):
3     # Return objective expression from the refined mathematical model
4     return sum(...)
5 model.objective = pyo.Objective(rule=objective_rule, sense=pyo.maximize)
6
7 # --- SOC Dynamics Constraint ---
8 def soc_dynamics_rule(model, t):
9     if t == model.T.first():
10         return model.e_soc[t] == params['E_soc_initial'] + ...
11     return model.e_soc[t] == model.e_soc[t-1] + ...
12 model.soc_dynamics = pyo.Constraint(model.T, rule=soc_dynamics_rule)
13
14 # --- Daily Cycle Constraint ---
15 def daily_cycle_rule(model, d):
16     # Get all time steps 't' for day 'd'
17     time_steps_in_day = [t for t in model.T if data.loc[t, 'day_id'] == d]
18     return sum(model.p_ch[t] * params['dt'] for t in time_steps_in_day) <= \
19         model.N_cycles * model.E_nom
20 model.daily_cycle_limit = pyo.Constraint(model.D, rule=daily_cycle_rule)
```

Listing 4: Example Constraint Implementation

4.4 Solve and Extract Results

- Instantiate a solver (e.g., CBC, GLPK) and call the `solve()` method.
- Extract the optimal values from the model's variables.
- Return a structured object (e.g., a dictionary) containing the key results for the scenario.

5 Post-Processing and Output Generation (main.py)

The final step aggregates results and formats them according to submission guidelines.

5.1 Aggregate Results and Calculate Metrics

- Convert the list of result dictionaries into pandas DataFrames.
- Calculate final metrics like Levelized Return on Investment (ROI) over a 10-year horizon.

5.2 Write to Excel

As per the FaQ, create three Excel files, each containing separate sheets for each country.

- TechArena_Phase1_Configuration.xlsx
- TechArena_Phase1_Investment.xlsx
- TechArena_Phase1_Operation.xlsx

```
1 # Pseudocode for writing one of the output files
2 with pd.ExcelWriter('output/TechArena_Phase1_Configuration.xlsx') as writer:
3     for country in countries:
4         df_config_country = df_all_configs[df_all_configs['Country'] == country]
5         df_config_country.to_excel(writer, sheet_name=country, index=False)
6 # Repeat for Investment and Operation files
```

Listing 5: Pseudocode for Writing Excel Output